

Homework 05

Author: Manushri Pendekanti (mp79)

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import optimize

%matplotlib inline
```

Question 1: Estimate the bias and variance of KNN

Question 1 — Part a

(Your written answer.)

```
In [2]: import numpy as np
from sklearn.neighbors import KNeighborsRegressor

np.random.seed(1)

n = 400
x0 = np.array([[0.5, 0.7, 1]])
k_values = np.arange(1, 30, 2)

true_mean = 0.5 * x0[0, 0] + np.sin(x0[0, 1]) - 0.3 * x0[0, 2]**2
print("f(x0) =", true_mean)

X = np.random.normal(size=(n, 3))
y = (0.5 * X[:, 0] + np.sin(X[:, 1]) - 0.3 * X[:, 2]**2
      + np.random.normal(0, 0.1, n))

for k in k_values:
    knn_fit = KNeighborsRegressor(n_neighbors=k, weights="uniform",
                                  algorithm="brute")
    knn_fit.fit(X, y)
    print("K =", k, "Prediction =", knn_fit.predict(x0)[0])
```

```

f(x0) = 0.594217687237691
K = 1 Prediction = 0.27673827338335283
K = 3 Prediction = 0.2896058985084401
K = 5 Prediction = 0.42966896277745725
K = 7 Prediction = 0.35179660395254747
K = 9 Prediction = 0.3562715742285425
K = 11 Prediction = 0.31061927506587994
K = 13 Prediction = 0.3947921231116194
K = 15 Prediction = 0.43559810340554733
K = 17 Prediction = 0.44417352120334946
K = 19 Prediction = 0.4795457782684412
K = 21 Prediction = 0.5128634654617371
K = 23 Prediction = 0.48824499723078935
K = 25 Prediction = 0.5361994510743727
K = 27 Prediction = 0.5350616072433807
K = 29 Prediction = 0.5409314045691525

```

Question 1 — Part b

(Your written answer.)

```

In [3]: import matplotlib.pyplot as plt

nsim = 200
predictions = np.full((nsim, len(k_values)), np.nan)

for l in range(nsim):
    X = np.random.normal(size=(n, 3))
    y = (0.5 * X[:, 0] + np.sin(X[:, 1]) - 0.3 * X[:, 2])**2
        + np.random.normal(0, 0.1, n))

    for j, k in enumerate(k_values):
        knn_fit = KNeighborsRegressor(n_neighbors=k, weights="uniform",
                                      algorithm="brute")

        knn_fit.fit(X, y)
        predictions[l, j] = knn_fit.predict(x0)[0]

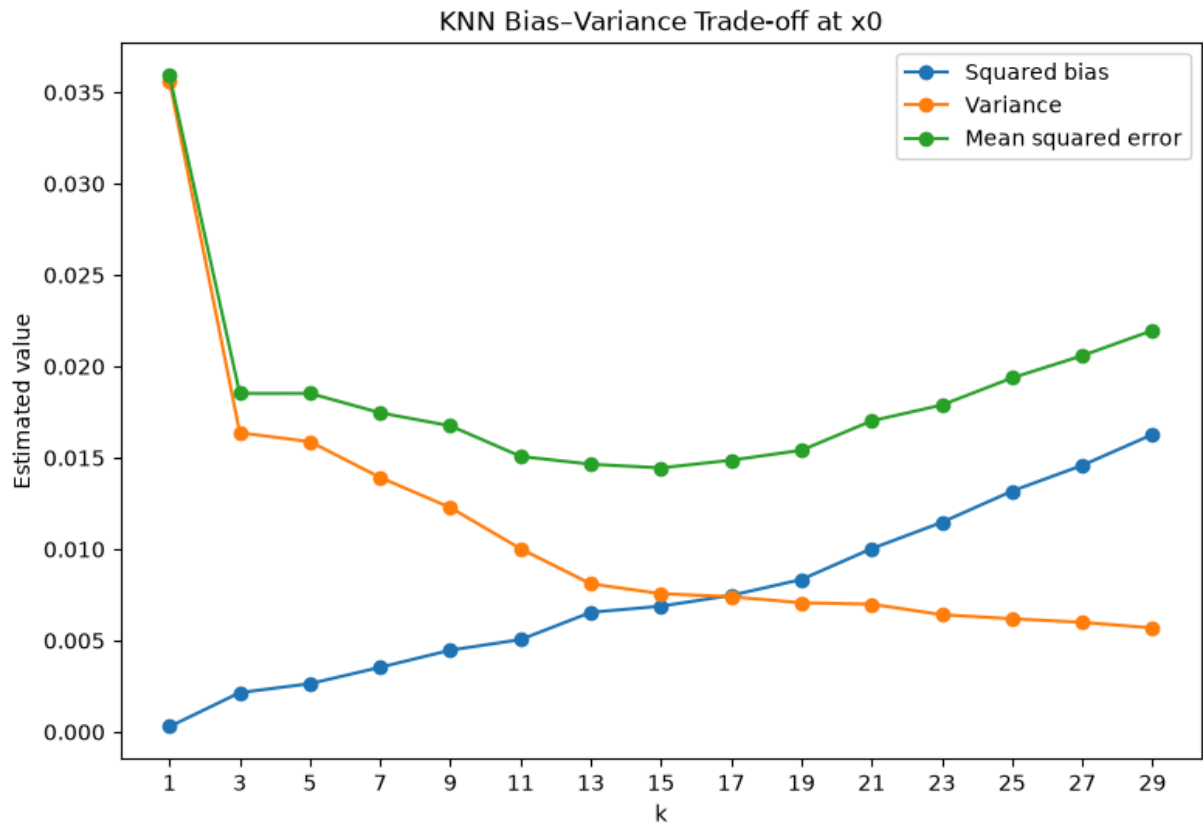
mean_prediction = np.mean(predictions, axis=0)
bias = mean_prediction - true_mean
squared_bias = bias**2

variance = np.mean((predictions - mean_prediction)**2, axis=0)
mse = np.mean((predictions - true_mean)**2, axis=0)

plt.figure(figsize=(9, 6))
plt.plot(k_values, squared_bias, marker="o", label="Squared bias")
plt.plot(k_values, variance, marker="o", label="Variance")
plt.plot(k_values, mse, marker="o", label="Mean squared error")
plt.xlabel("k")
plt.ylabel("Estimated value")
plt.title("KNN Bias-Variance Trade-off at x0")
plt.xticks(k_values)
plt.legend()
plt.show()

```

#An increase in k leads to an average increase
 #in number of responses used for training, thereby leading to a decrease in
 #However, an increase in k means that we use data further from x_0 ,
 #which ncreasing bias. The MSE decreases initially and then increases due to



Question 2: Compare KNN with lasso

Question 2 — Part a

(Your written answer.)

```
In [4]: from sklearn.linear_model import Lasso
from sklearn.model_selection import KFold, GridSearchCV
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

np.random.seed(1)

n = 400
ntest = 1000
p = 30
penalties = np.logspace(-4, 0, 41)

for setting in [1, 2]:
    if setting == 1:
```

```

X = np.random.normal(size=(n, p))
test_X = np.random.normal(size=(ntest, p))
else:
    Sigma = np.full((p, p), 0.8)
    np.fill_diagonal(Sigma, 1)

    X = np.random.multivariate_normal(np.zeros(p), Sigma, size=n)
    test_X = np.random.multivariate_normal(np.zeros(p), Sigma, size=ntest)
y = (0.5 * X[:, 0] + np.sin(X[:, 1]) - 0.3 * X[:, 2])**2
    + np.random.normal(0, 0.1, n)
test_y = (0.5 * test_X[:, 0] + np.sin(test_X[:, 1])
          - 0.3 * test_X[:, 2])**2
          + np.random.normal(0, 0.1, ntest))
knn_fit = KNeighborsRegressor(n_neighbors=5, weights="uniform",
                             algorithm="brute")

knn_fit.fit(X, y)
knn_mse = np.mean((knn_fit.predict(test_X) - test_y)**2)
folds = KFold(n_splits=10, shuffle=True, random_state=1)
lasso_cv = GridSearchCV(
    make_pipeline(
        StandardScaler(),
        Lasso(fit_intercept=True, max_iter=100000)
    ),
    {"lasso__alpha": penalties},
    cv=folds,
    scoring="neg_mean_squared_error"
)
lasso_cv.fit(X, y)
lasso_fit = lasso_cv.best_estimator_
lasso_mse = np.mean((lasso_fit.predict(test_X) - test_y)**2)
selected_penalty = lasso_cv.best_params_["lasso__alpha"]
nonzero_slopes = np.count_nonzero(lasso_fit.named_steps["lasso"].coef_)

print(setting, knn_mse, lasso_mse, selected_penalty, nonzero_slopes)

```

```

1 0.5721475535296535 0.2486324395956727 0.01584893192461114 19
2 0.18350233044427633 0.27123411601834063 0.01584893192461114 11

```

Question 2 — Part b

(Your written answer.)

```

In [5]: #For independent predictors, the distance used by KNN includes 27 covariates
#The use of correlation increased the informativeness of the neighborhoods i
#As opposed to KNN where each covariate is needed to form neighborhoods, Las

```

Question 2 — Part c

(Your written answer.)

```
In [6]: #The lasso is based on linear functions only; therefore, it will not be able  
#while the KNN is capable of capturing the non-linear patterns locally. The
```

Question 3: Observed dimension and latent dimension

Question 3 — Part a

(Your written answer.)

```
In [7]: np.random.seed(1)

p = 100
n = 200
ntest = 200
k_values = np.arange(2, 83, 8) # 2, 10, ..., 82

def generate_data(m):
    A = np.random.uniform(-2, 2, size=(p, m))

    Z = np.random.normal(size=(n, m))
    X = Z @ A.T + np.random.normal(size=(n, p))
    y = Z[:, 0] + Z[:, 1] + np.random.normal(size=n)

    test_Z = np.random.normal(size=(ntest, m))
    test_X = test_Z @ A.T + np.random.normal(size=(ntest, p))
    test_y = test_Z[:, 0] + test_Z[:, 1] + np.random.normal(size=ntest)

    return X, y, test_X, test_y

for m in [3, 30]:
    X, y, test_X, test_y = generate_data(m)

    for k in k_values:
        knn_fit = KNeighborsRegressor(n_neighbors=k, weights="uniform",
                                      algorithm="brute")

        knn_fit.fit(X, y)
        test_pred = knn_fit.predict(test_X)
        print("m =", m, "k =", k,
              "Test MSE =", np.mean((test_pred - test_y)**2))
```

```

m = 3 k = 2 Test MSE = 1.8992332108734302
m = 3 k = 10 Test MSE = 1.349602068895506
m = 3 k = 18 Test MSE = 1.4123066421637618
m = 3 k = 26 Test MSE = 1.407059326133276
m = 3 k = 34 Test MSE = 1.4571542437967038
m = 3 k = 42 Test MSE = 1.50575433556465
m = 3 k = 50 Test MSE = 1.574563625119955
m = 3 k = 58 Test MSE = 1.6426440502587079
m = 3 k = 66 Test MSE = 1.7077075723046864
m = 3 k = 74 Test MSE = 1.7681660436954374
m = 3 k = 82 Test MSE = 1.8247865856579313
m = 30 k = 2 Test MSE = 2.8390942809392046
m = 30 k = 10 Test MSE = 2.182358320220329
m = 30 k = 18 Test MSE = 2.2763326858295754
m = 30 k = 26 Test MSE = 2.2808494604368534
m = 30 k = 34 Test MSE = 2.2851656863999135
m = 30 k = 42 Test MSE = 2.3588983484180237
m = 30 k = 50 Test MSE = 2.4108106129527274
m = 30 k = 58 Test MSE = 2.4331301508796788
m = 30 k = 66 Test MSE = 2.471016667794854
m = 30 k = 74 Test MSE = 2.4785260655825048
m = 30 k = 82 Test MSE = 2.521360238234542

```

Question 3 — Part b

(Your written answer.)

```

In [8]: nsim = 50
all_mse = np.full((nsim, len(k_values), 2), np.nan)

for l in range(nsim):
    for j, m in enumerate([3, 30]):
        X, y, test_X, test_y = generate_data(m)

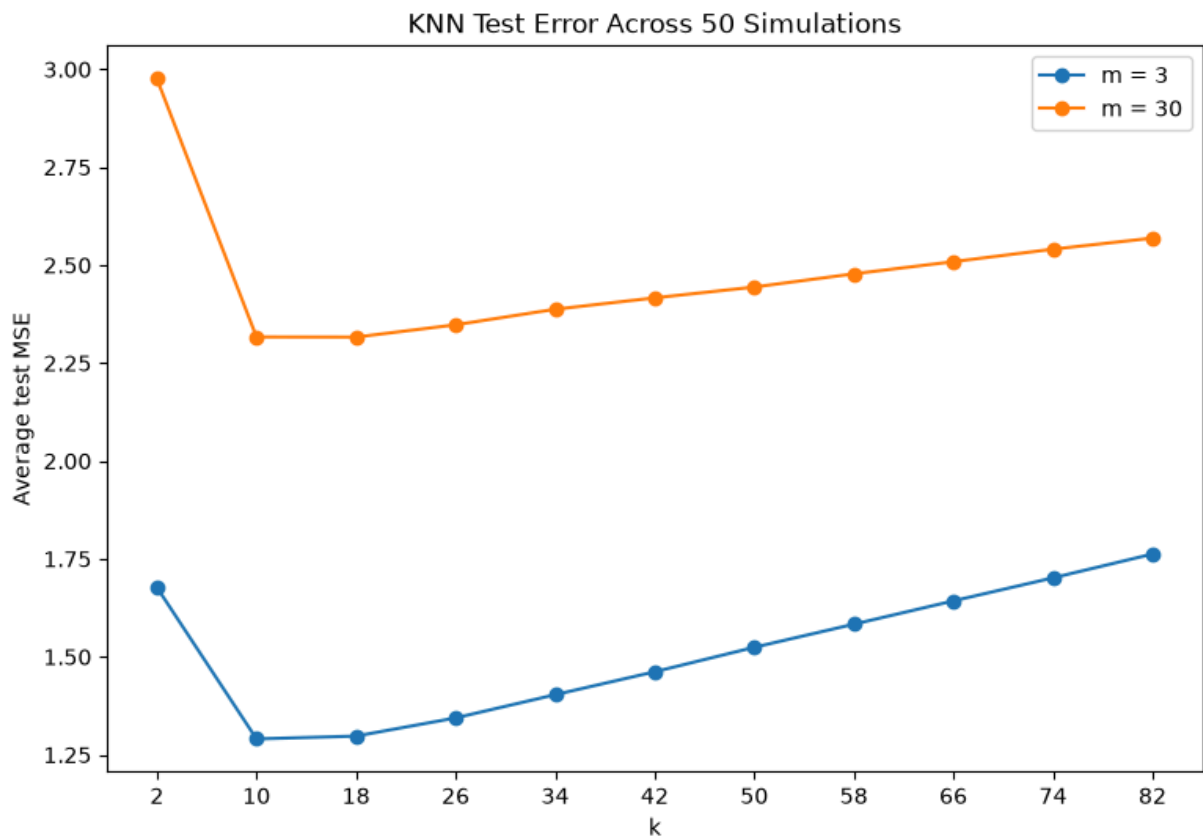
        for h, k in enumerate(k_values):
            knn_fit = KNeighborsRegressor(n_neighbors=k, weights="uniform",
                                         algorithm="brute")

            knn_fit.fit(X, y)
            test_pred = knn_fit.predict(test_X)
            all_mse[l, h, j] = np.mean((test_pred - test_y)**2)

plt.figure(figsize=(9, 6))
for j, m in enumerate([3, 30]):
    plt.plot(k_values, np.mean(all_mse[:, :, j], axis=0),
             marker="o", label=f"m = {m}")

plt.xlabel("k")
plt.ylabel("Average test MSE")
plt.title("KNN Test Error Across 50 Simulations")
plt.xticks(k_values)
plt.legend()
plt.show()

```



Question 3 — Part c

(Your written answer.)

```
In [9]: #For(m=30), variations in 28 latent variables that have no effect on (Y) can
#in (X). For (m=3), Euclidean neighbors are more likely to be alike in the t
#Noise (noise_i) further hides any such similarity since the noise is added
#so that (X_i) do not lie precisely in the m-dimensional space of (A_Z).
```

Question 4: Classify handwritten digits

Question 4 — Part a

(Your written answer.)

```
In [10]: from pathlib import Path

zip_train = np.loadtxt(Path("data/knn/zip-train.csv"), delimiter=",")[:1000]
zip_test = np.loadtxt(Path("data/knn/zip-test.csv"), delimiter=",")[:500]

train_y = zip_train[:, 0].astype(int)
train_X = zip_train[:, 1:] # Exclude digit labels from distance
test_y = zip_test[:, 0].astype(int)
```

```

test_X = zip_test[:, 1:]

def classify_5nn(train_X, train_y, test_X, distance):
    predictions = np.empty(len(test_X), dtype=int)

    for j, x0 in enumerate(test_X):
        if distance == "euclidean":
            # No square root needed: it would not change the ordering.
            distances = np.sum((train_X - x0)**2, axis=1)
        elif distance == "manhattan":
            distances = np.sum(np.abs(train_X - x0), axis=1)
        else:
            raise ValueError("Choose 'euclidean' or 'manhattan'.")

        nearest = np.argsort(distances, kind="stable")[:5]
        votes = np.bincount(train_y[nearest], minlength=10)
        predictions[j] = np.argmax(votes)

    return predictions

```

Question 4 — Part b

(Your written answer.)

```

In [11]: euclidean_pred = classify_5nn(train_X, train_y, test_X, "euclidean")
         manhattan_pred = classify_5nn(train_X, train_y, test_X, "manhattan")

         for name, pred in [("Euclidean", euclidean_pred),
                             ("Manhattan", manhattan_pred)]:
             print(name, "test error:", np.mean(pred != test_y))
             print(pd.crosstab(
                 pd.Categorical(pred, categories=range(10)),
                 pd.Categorical(test_y, categories=range(10)),
                 rownames=["Prediction"],
                 colnames=["Observed"],
                 dropna=False
             ))

```

Euclidean test error: 0.122

Observed	0	1	2	3	4	5	6	7	8	9
Prediction										
0	118	0	7	1	0	4	2	0	2	0
1	0	66	0	0	2	0	0	0	0	1
2	0	0	39	1	0	0	2	0	0	1
3	0	0	0	23	0	4	0	0	2	0
4	0	0	1	0	27	1	1	1	0	1
5	0	0	0	1	0	13	0	0	0	0
6	0	2	0	0	0	1	41	0	2	0
7	0	0	0	0	1	1	0	34	0	4
8	0	0	3	2	0	2	0	0	37	0
9	1	0	0	1	4	0	0	1	1	41

Manhattan test error: 0.138

Observed	0	1	2	3	4	5	6	7	8	9
Prediction										
0	118	0	6	1	0	5	3	0	4	0
1	0	67	2	0	2	0	1	0	0	1
2	0	0	35	0	0	0	2	0	0	0
3	0	0	1	24	0	4	0	0	2	0
4	0	0	1	1	27	1	0	1	0	1
5	0	0	0	1	0	11	0	0	0	0
6	0	1	0	0	0	2	40	0	2	0
7	0	0	2	1	1	0	0	34	0	5
8	0	0	2	0	0	3	0	0	34	0
9	1	0	1	1	4	0	0	1	2	41

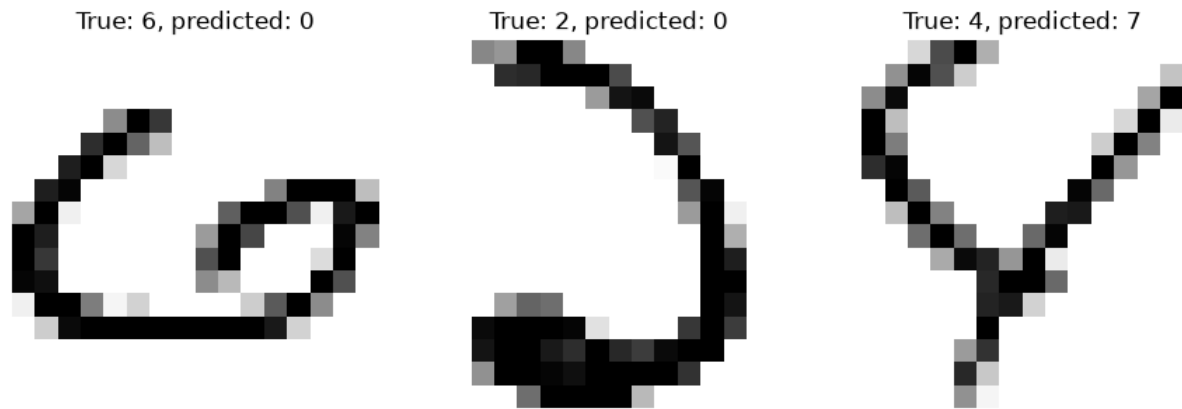
Question 4 — Part c

(Your written answer.)

```
In [12]: wrong = np.flatnonzero(euclidean_pred != test_y)

plt.figure(figsize=(9, 3))
for j, i in enumerate(wrong[:3], start=1):
    plt.subplot(1, 3, j)
    plt.imshow(test_X[i].reshape(16, 16), cmap="gray_r",
               vmin=0, vmax=1, interpolation="nearest")
    plt.title(f"True: {test_y[i]}, predicted: {euclidean_pred[i]}")
    plt.axis("off")
plt.tight_layout()
plt.show()

print("Different predictions:", np.sum(euclidean_pred != manhattan_pred))
print("Euclidean wrong, Manhattan right:",
      np.sum((euclidean_pred != test_y) & (manhattan_pred == test_y)))
print("Manhattan wrong, Euclidean right:",
      np.sum((manhattan_pred != test_y) & (euclidean_pred == test_y)))
```



Different predictions: 22
 Euclidean wrong, Manhattan right: 3
 Manhattan wrong, Euclidean right: 11

Question 5: Create a plot-formatting skill

(Notes about creating, testing, and using the skill.)

--- name: stat432-plot-formatting description: Improves formatting of STAT 432 homework plots produced in R or Python. Use only when the user explicitly asks to use this skill on a homework figure. disable-model-invocation: true --- # STAT 432 Homework Plot Formatting ## Intent Polish homework figures for readability and submission quality without changing the underlying data, statistics, or analysis. ## When to use Apply this skill only when the student explicitly names `stat432-plot-formatting` or asks you to use this plot-formatting skill on a homework figure. ## Formatting rules ### Margins and layout - Call `tight\_layout()` (Python) or use adequate theme margins (R) so titles and labels are not clipped. - Default figure size about `(8, 5)` for one panel; `(10, 6)` for multi-series line plots. - Keep legends inside the plot area only when they do not cover important data. ### Titles - Use one clear, descriptive title in sentence case. - State what is being plotted and the main comparison (for example, error components versus a tuning parameter). ### Axes - Use tick marks that match the data (every value, every other value, or a regular grid). - Start the y-axis at 0 when all plotted values are non-negative and the scale is absolute. - Rotate x-axis labels slightly if tick labels overlap. ### Labels - Write full axis labels, not single-letter shortcuts alone. - Name the x-variable with its role (for example, “Number of neighbors (k)”). - Name the y-variable with what is estimated or measured (for example, “Estimated squared bias, variance, and MSE”). - Include units when the quantity has them. ### Colors - Prefer a colorblind-friendly palette (for example, matplotlib `tab10` or a ColorBrewer set). - Assign a distinct color to each series and keep legend order matched to the plot order. - Combine color with line style or markers so series remain distinguishable in grayscale. ### Sizes - Base text around 11–12 pt; title around 13–14 pt. - Line width about 1.5–2; marker size about 5–7. - Keep legend text the same size as axis labels. ## Rules - Do not change data, simulation settings, model fits, or numerical results. - Do not refer to specific homework question numbers inside the skill file. - Apply the same standards in matplotlib/seaborn (Python) or ggplot2 (R). ## Plot-formatting skill * (Paste the complete contents of your final plot-formatting `SKILL.md` below.)*